

general, any name conforming to these rules MAY be used, as long as it does not collide with a [key-word](#) used by the schema language.

B.1.4. Namespaces

The name assigned to a schema construct SHALL be unique relative to all other named constructs in the same scope. To facilitate this, TLV Schemas support a namespacing mechanism similar to that provided in languages like C++.

The names of constructs defined within a [namespace](#) definition are only required to be unique within the given namespace. Namespaces themselves MAY be nested to any depth.

Constructs defined in other namespaces MAY be referenced using a name that gives the enclosing namespaces, plus the construct name, each separated by dots (.). Such a multi-part name is called a *scoped name*. For example:

Namespaces Example

```
namespace a
{
    x => STRING,
    other-x => b.x
}

namespace b
{
    x => SIGNED INTEGER
}
```

See [namespace-def](#) for further details.

B.1.5. Qualifiers

Constructs within a TLV Schema MAY be annotated with additional information using a *qualifier*. Qualifiers appear within square brackets ([...]) immediately following the construct they affect. In most cases the use of qualifiers is optional, but there are some situations where the schema syntax requires a qualifier.

Often qualifiers are used to place restrictions on the form or range of values that a construct can assume. For example a [length](#) qualifier MAY be used to constrain the length of a STRING type:

```
international-standard-book-number => STRING [length 13]
```

Multiple qualifiers MAY appear within the square brackets, and SHALL be separated by commas.

See [Section B.5, “Qualifiers”](#) for further details.